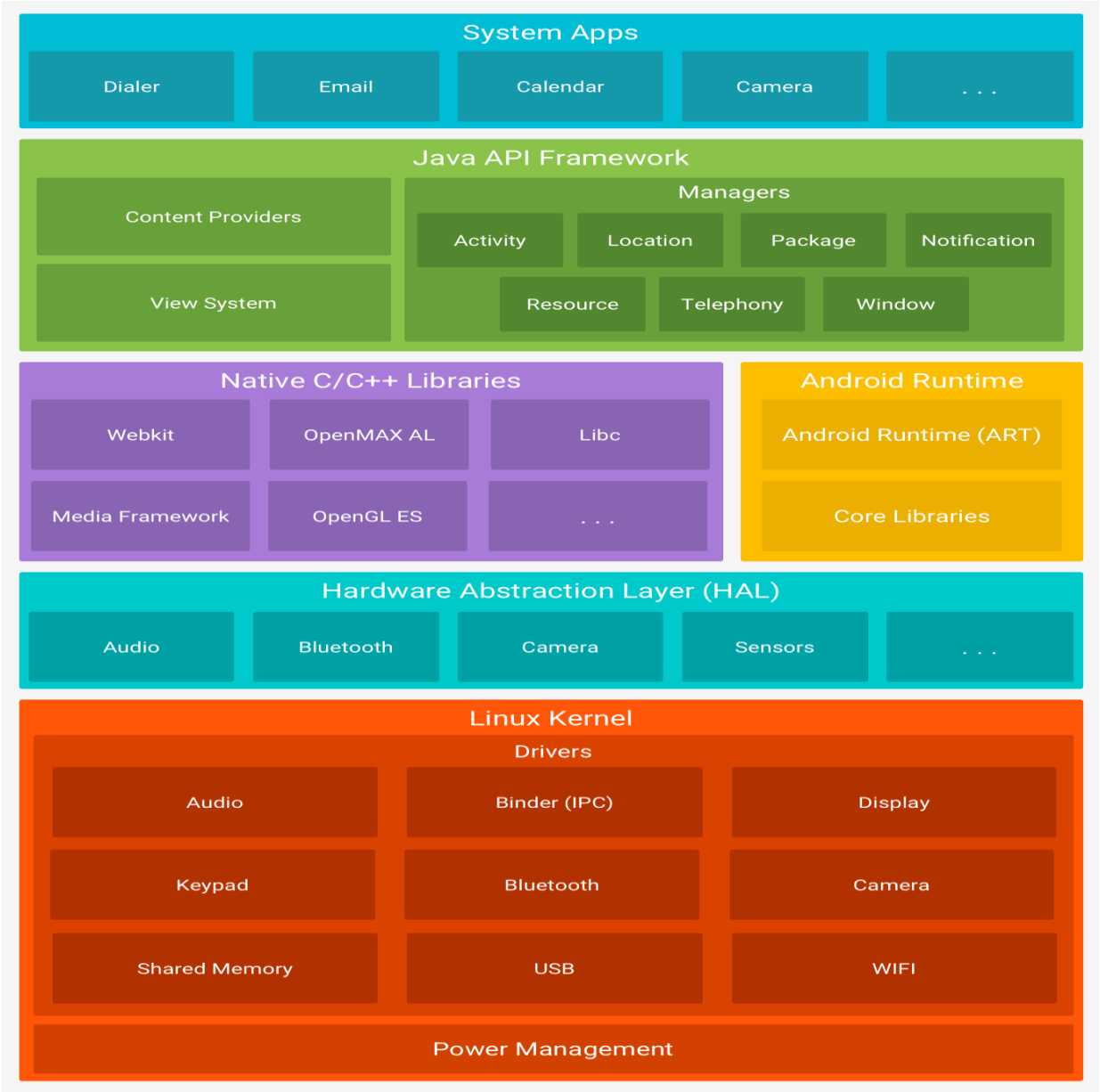
Android Architecture

M2P - IG

Ahmad Shahin

Layering example: Android Architecture

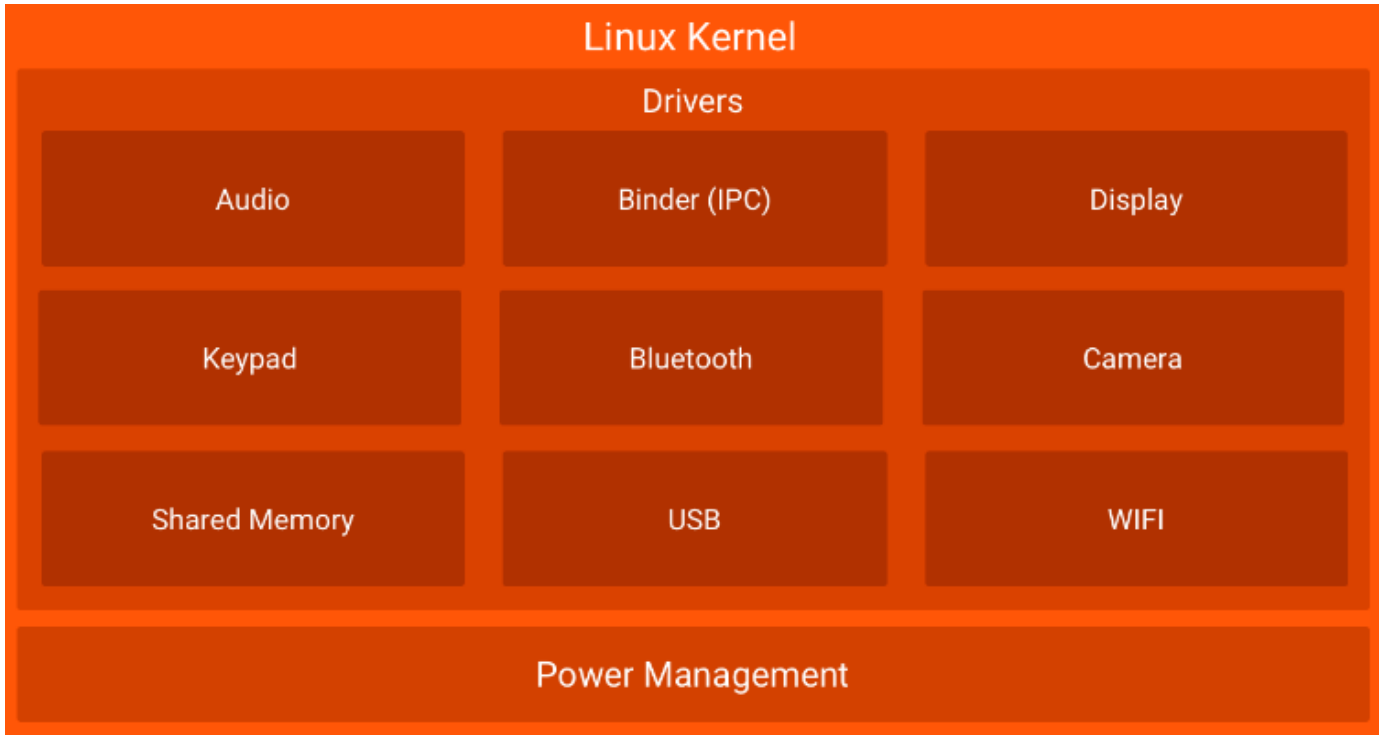
- Android is
  - open source
  - Linux-based software stack
  - created for a wide array of devices
  - major components



Overview of Android Layers

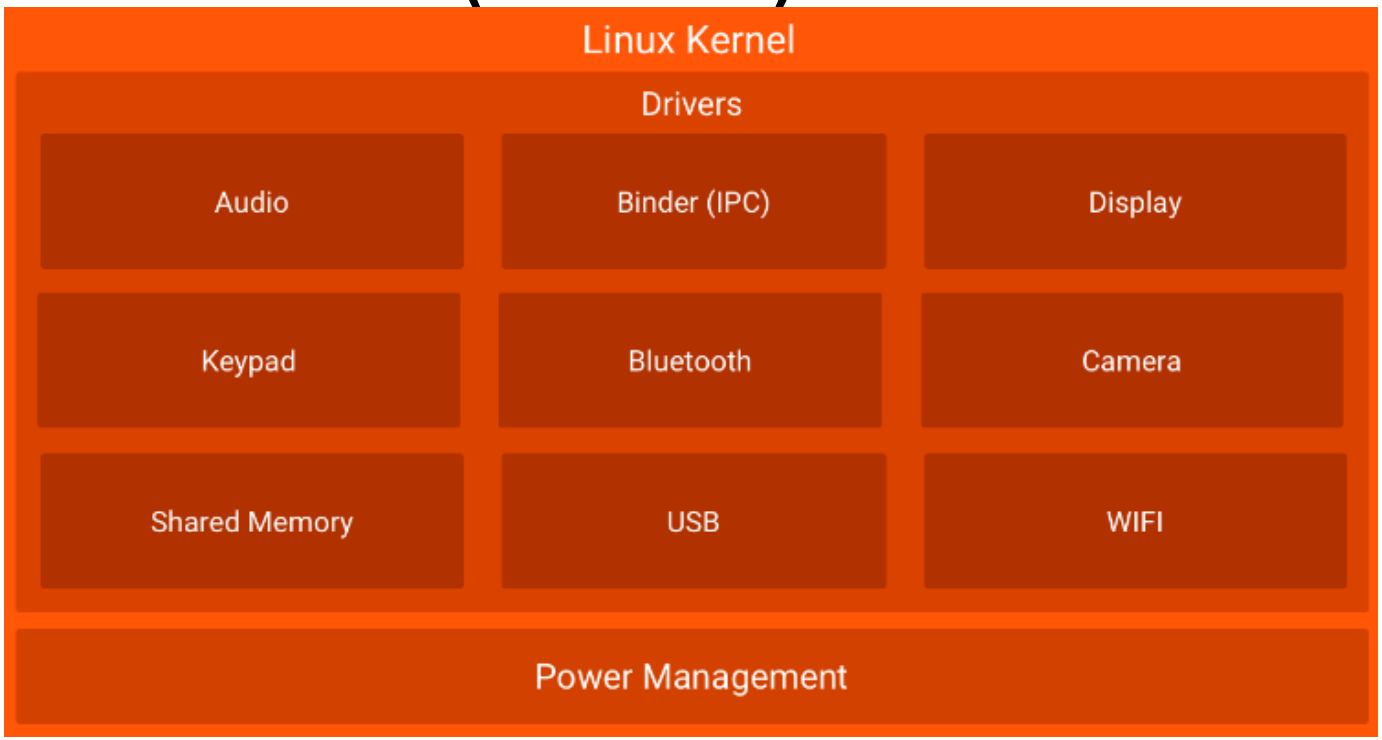
- Part 1
  - https://www.youtube.com/watch?v=Thl_0ycg-i8
  - About 26'24“: overview of key layers in the Android software architecture, focusing on the Android Linux layer, the Hardware Abstraction Layer, and the Virtual Machine layer.
- Part 2
  - https://www.youtube.com/watch?v=-XsdClm9yAM
  - About 24'11“: presents the second half of the overview of layers in the Android software stack: core libraries, native libraries, API frameworks…

Android S/W Stack – Linux Kernel



- Relying on Linux Kernel 2.6 for core system services
  - ✓ Low-level Memory Management
  - ✓ Thread and Process Management
  - ✓ Network Stack
  - ✓ Power management
  - ✓ …

Android S/W Stack – Linux Kernel (Cont'd)

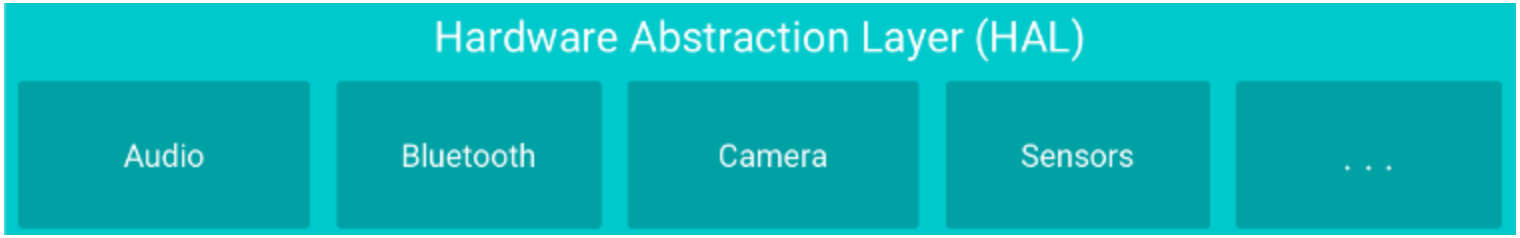


- Using a Linux kernel allows Android to take advantage of key security features
- It allows device manufacturers to develop hardware drivers for a well-known kernel.

Android Linux Kernel

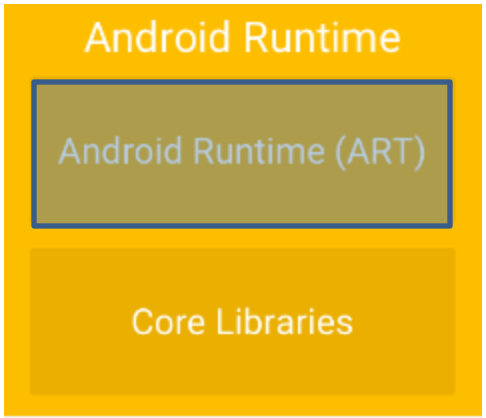
- Part 1
  - https://www.youtube.com/watch?v=luMlYPxM8wc
  - About 15': overviewing Primary and Secondary Storage Mechanisms
- Part 2
  - https://www.youtube.com/watch?v=MxDstgPXqvs
  - About 19': presenting the Core Kernel IPC and Processing Mechanisms
- Part 3
  - https://www.youtube.com/watch?v=cTNcCMp4DRc
  - About 13': highlighting the Android Kernel Extensions

Android S/W Stack – HAL



- Hardware abstraction layer (HAL)
  - ✓ provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework
  - ✓ consists of multiple library modules, each of which implements an interface for a specific type of hardware component,
    - ➢ such as the camera or bluetooth module…
  - ✓ When a framework API makes a call to access device hardware, the Android system loads the library module for that hardware component.
  - ✓ https://www.youtube.com/watch?v=dHXJA8D8Zf0
    - ➢ About 11‘: presenting HAL

Android S/W Stack - Runtime



- Each app runs in its own process and with its own instance of the Android Runtime (ART) also called virtual machine
  - https://www.youtube.com/watch?v=CKax4-Y8FyE
  - About 10‘ presenting Android Runtime Execution Environment
- For devices running Android version 5.0 (API level 21) or higher (Dalvik is the Former ART)
- ART is written to run multiple virtual machines on low-memory devices by executing DEX files:
  - a bytecode format designed specially for Android that's optimized for minimal memory footprint
  - Build toolchains, such as Jack, compile Java sources into DEX bytecode, which can run on the Android platform

Android S/W Stack – Runtime (Cont'd)

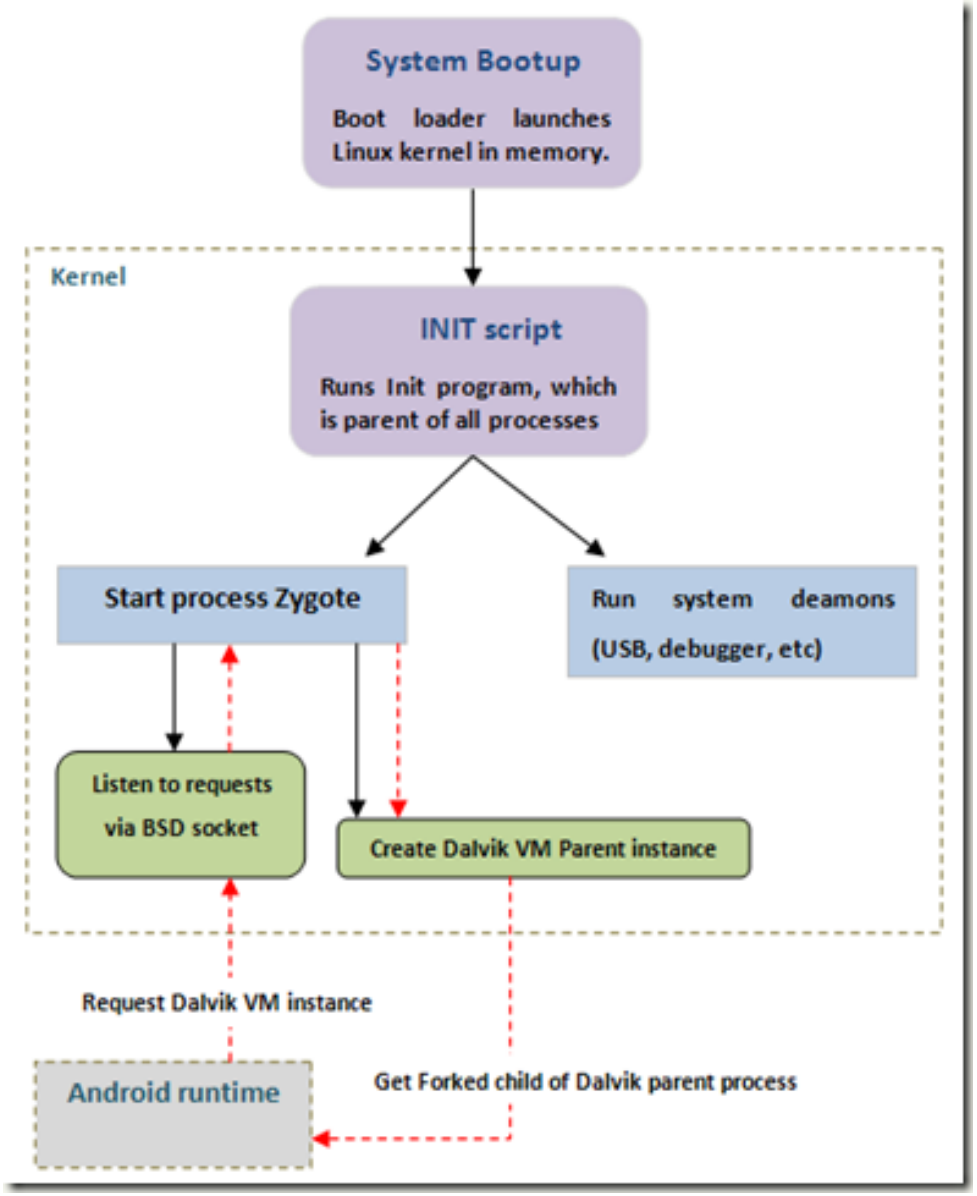
- Some of the major features of ART include the following:
  - Ahead-of-time (AOT) and just-in-time (JIT – Dalvik mechanism) compilation
  - Optimized garbage collection (GC)
  - Better debugging support
- Android apps were previously deployed in Dalvik bytecode, which is portable, unlike native code.
- In order to be able to run the app on a device, the code has to be compiled to machine code.
- ART
  - ✓ Register-based virtual machine (See example)

Android S/W Stack – Runtime (Cont'd)

- Dalvik is based on **JIT (just in time) compilation**.
  - It means that each time you run an app, the part of the code required for its execution is going to be translated (compiled) to machine code at that moment.
  - As you progress through the app, additional code is going to be compiled and cached, so that the system can reuse the code while the app is running. Since JIT compiles only a part of the code, it has a smaller memory footprint and uses less physical space on the device.
- ART, on the other hand, compiles the intermediate language, Dalvik bytecode, into a **system-dependent binary**.
  - The whole code of the app will be pre-compiled during install (once), thus removing the lag that we see when we open an app on our device.
  - With no need for JIT compilation, the code should execute much faster.

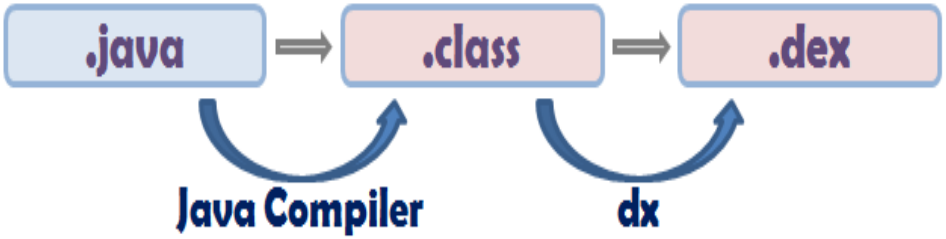
Android S/W Stack – Runtime (Cont'd)

- Dalvik Virtual Machine (Cont'd)
- When the system boots up, the boot loader loads the kernel into memory and initializes system parameters. Soon after this,
  - The kernel runs the Init program, which is the parent process for all processes in the system.
  - The Init program starts system daemons and the very important 'Zygote' service.
  - The Zygote process creates a Dalvik instance which will be the parent Dalvik process for all Dalvik VM instances in the system.
  - The Zygote process also sets up a BSD read socket and listens for incoming requests.
  - When a new request for a Dalvik VM instance is received, the Zygote process forks the parent Dalvik VM process and sends the child process to the requesting application.



Android S/W Stack – Runtime (Cont'd)

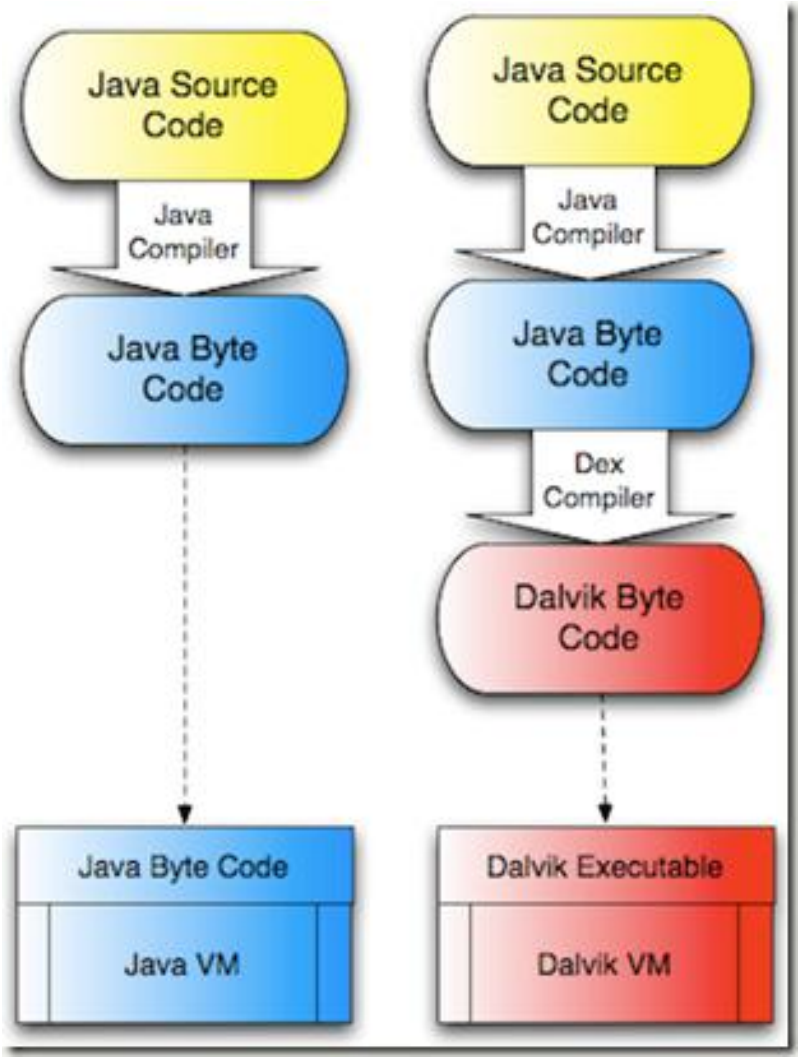
- Dalvik Virtual Machine (Cont'd)
  - ✓Executing the Dalvik Executable (.dex) format
    - ➢.dex format is optimized for minimal memory footprint.
    - ➢Compilation



  - ✓Relying on the Linux Kernel for:
    - ➢Threading
    - ➢Low-level memory management

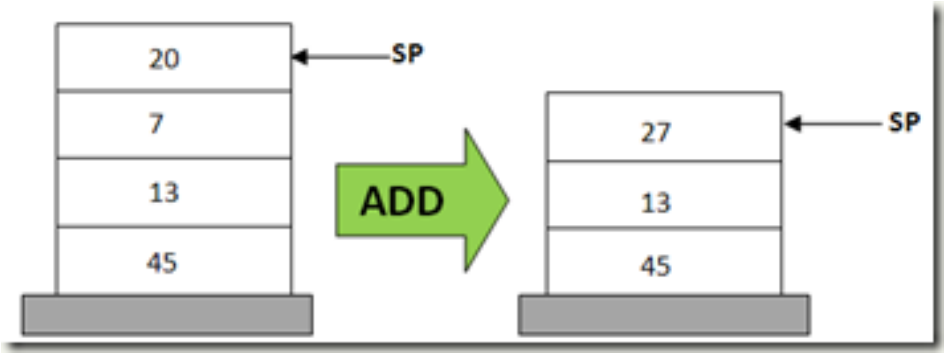
Android S/W Stack – Runtime (Cont'd)

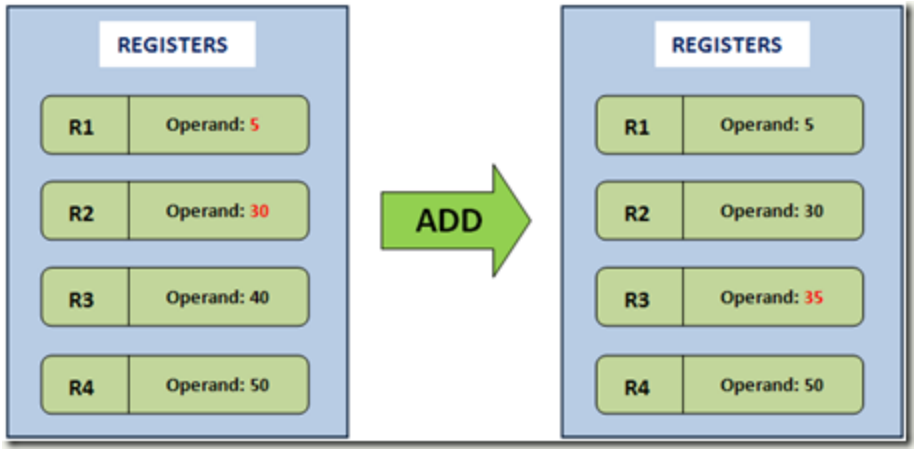
- Java VM vs Dalvik VM
  - The DEX compiler converts the java .class file into a .dex file, which is of less size and more optimized for the Dalvik VM.



Android S/W Stack – Runtime (Cont'd)

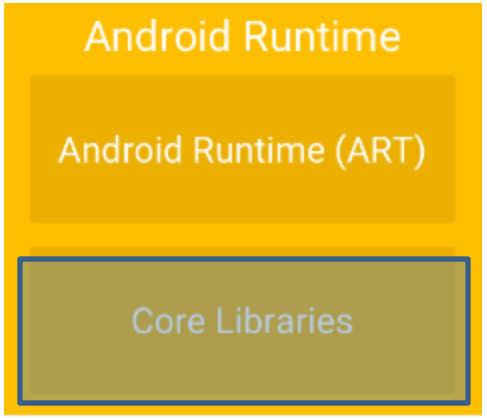
- Stack Based Virtual Machines (Memory)
  - POP 20
  - POP 7
  - ADD 20, 7, result
  - PUSH result
- Register Based Virtual Machines
  - ADD R1, R2, R3 ;
  - #Add contents of R1 and R2, store result in R3



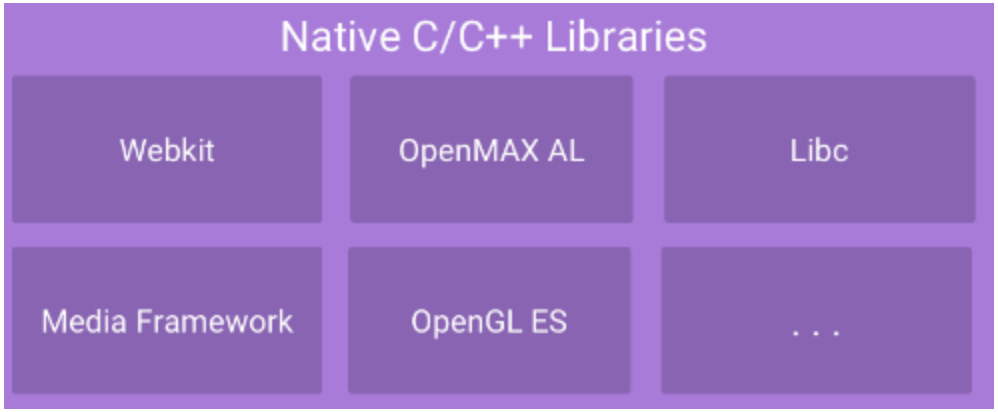


Android S/W Stack – Runtime (Cont'd)

- Core Libraries
  - ✓ Providing most of the functionality available in the core libraries of the Java language
    - ✓ Java.*, javax.*
    - ✓ Threading, Synchronization..
  - ✓ Android.* Classes
  - ✓ Android Concurrency Framework
    - ✓ AsyncTask
    - ✓ Handler, Messages, & Runnable s(HaMeR)
  - ✓ Android Services Framework
  - ✓ APIs
    - ➢ Data Structures
    - ➢ Utilities
    - ➢ File Access
    - ➢ Network Access
    - ➢ Graphics
    - ➢ Etc
  - ➢ Android Runtime Core Libraries
    - ➢ https://www.youtube.com/watch?v=xFd2rt7GZuo

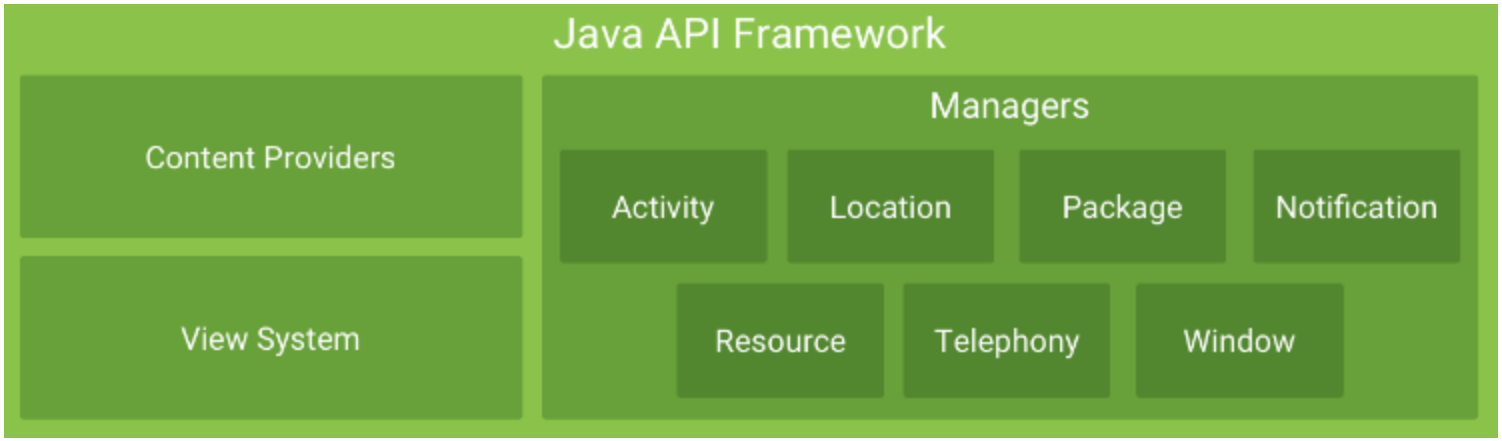


Android S/W Stack - Libraries



- Many core Android system components and services, such as ART and HAL, are built from native code that require native libraries written in C and C++. The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to apps.
  - For example, OpenGL ES is accessible through the Android framework's Java OpenGL API to add support for drawing and manipulating 2D and 3D graphics in your app.
- **JNI** is a native programming interface. It allows Java code that runs inside a Java Virtual Machine (VM) to interoperate with applications and libraries written in other programming languages, such as C, C++, and assembly.
- The Native Development Kit (**NDK**) is a set of tools that allows you to access (through JNI) some of these native platform libraries directly (use C and C++ code) with **Android**, and to manage native activities and access physical device components, such as sensors and touch input…
- Native Libraries
  - https://www.youtube.com/watch?v=xFd2rt7GZuo

Android S/W Stack – App Framework



- Set of API's that allow developers to quickly and easily write apps
- These APIs form the building blocks you need to create Android apps by simplifying the reuse of core, modular system components and services

Android S/W Stack – App Framework (Cont'd)

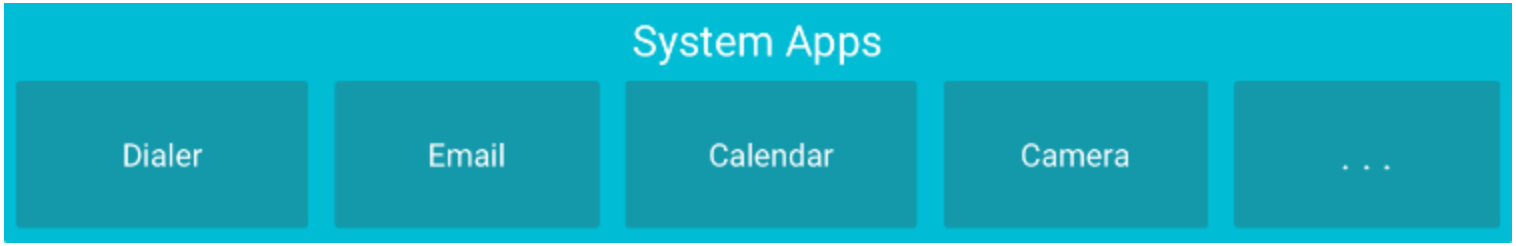
- Android app consists of
  - Activities (programs that the user interacts with),
  - services (programs that run in the background or provide some function to other apps),
  - and broadcast receivers (programs that catch information important to your app).
- Enabling and simplifying the reuse of components
  - ✓ Developers have full access to the same framework APIs used by the core applications.
  - ✓ Users are allowed to replace components.

Android S/W Stack – App Framework (Cont'd)

- Features

Feature	Role
View System	Used to build an application, including lists, grids, text boxes, buttons, and embedded web browser
Content Provider	Enabling applications to access data from other applications or to share their own data
Resource Manager	Providing access to non-code resources (localized strings, graphics, and layout files)
Notification Manager	Enabling all applications to display customer alerts in the status bar
Activity Manager	Managing the lifecycle of applications and providing a common navigation backstack
Window Manager	System service, which is responsible for managing the z-ordered list of windows, which windows are visible, and how they are laid out on screen.

Android S/W Stack – System Apps



- Android provides a set of core applications:
  - ✓ Email Client
  - ✓ SMS Program
  - ✓ Calendar
  - ✓ Maps
  - ✓ Browser
  - ✓ Contacts
  - ✓ Etc

- All applications are written using the Java language